



Congratulations!

You've completed Orchestrate complex workflows!

You now know how to choose the right workflow patterns, manage state in multi-agent systems, and understand when you might need custom agents or distributed systems

What you learned

Part 1: Choosing the right workflow

- Choose between sequential, parallel, and loop agents
- Coordinate state in parallel workflows with distinct keys
- Apply the aggregation pattern

Python

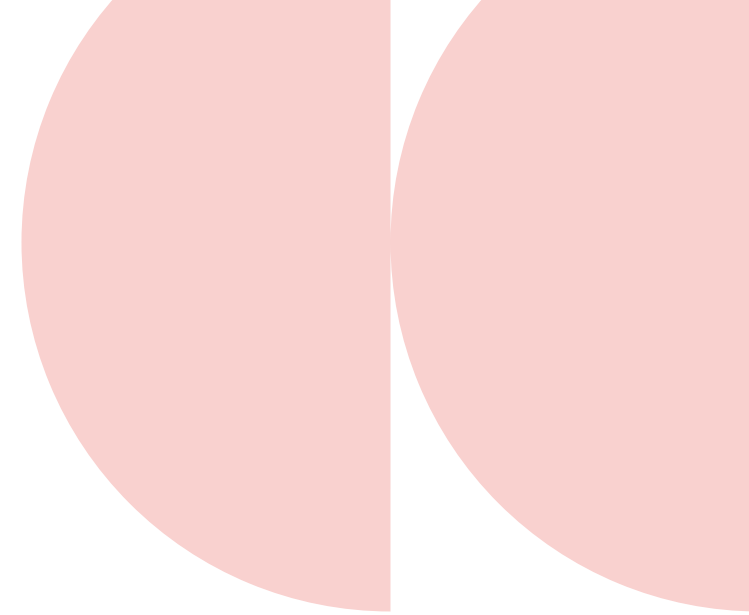
The essential pattern

```
parallel = ParallelAgent(sub_agents=[  
    LlmAgent(output_key='result_a', ...),  
    LlmAgent(output_key='result_b', ...)  
])
```

```
aggregator = LlmAgent(instruction='Use {result_a} and {result_b}')
```

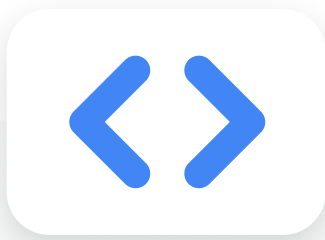
```
workflow = SequentialAgent(sub_agents=[parallel, aggregator])
```





Part 2: State management in workflows

- How state flows through workflow pipelines
- Using lesson 4 namespaces in workflows
- Session state for results, user: for preferences



```
Python
# State flow in workflows
agent_a = LlmAgent(output_key='data', ...) # Writes to state
agent_b = LlmAgent(instruction='Use {data}') # Reads from state
```

Part 3: When you need custom agents

- Workflow agents (Sequential/Parallel/Loop) cover most cases
- Custom agents are for conditional skipping or dynamic routing
- Default to workflow agents—custom agents are rare

Part 4: Distributed agent systems

- A2A Protocol enables cross-service agent communication
- Use for microservices, cross-team, different languages
- Most apps only need local agents

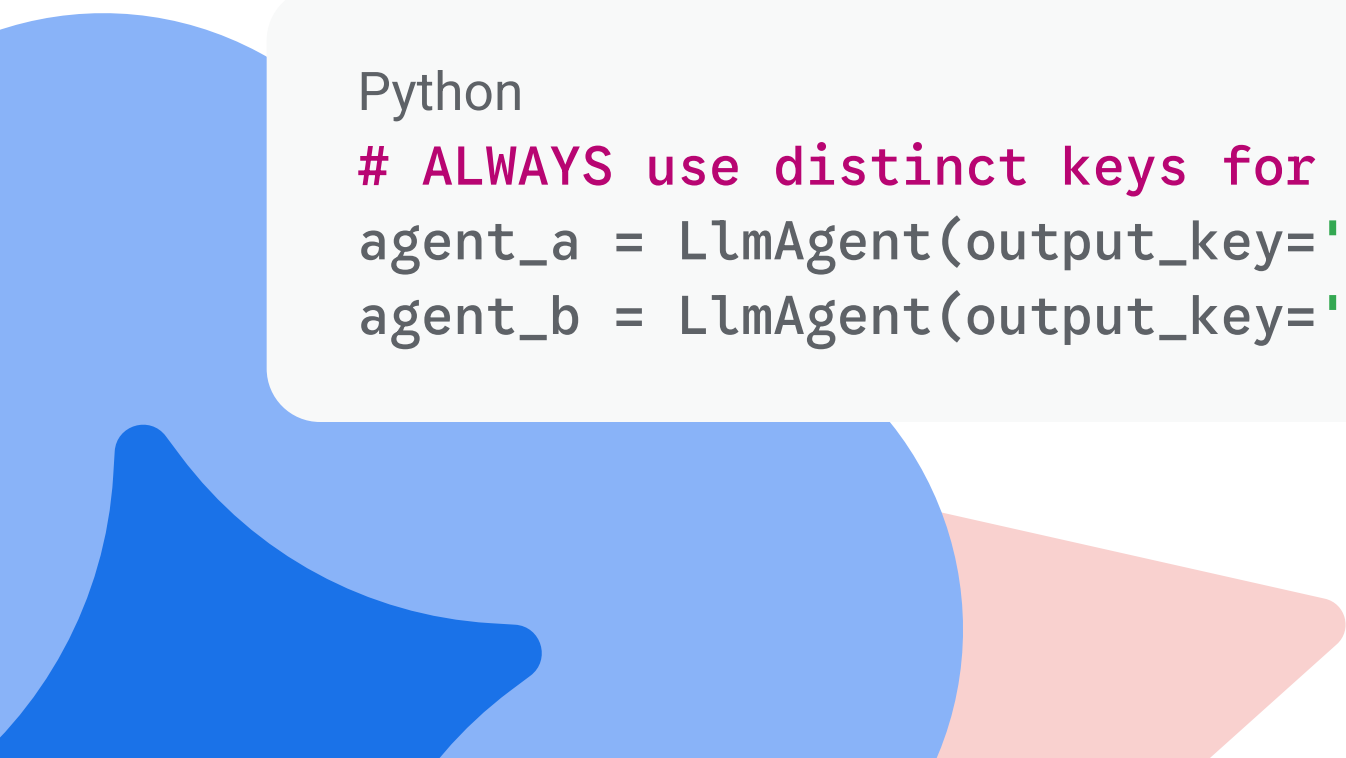
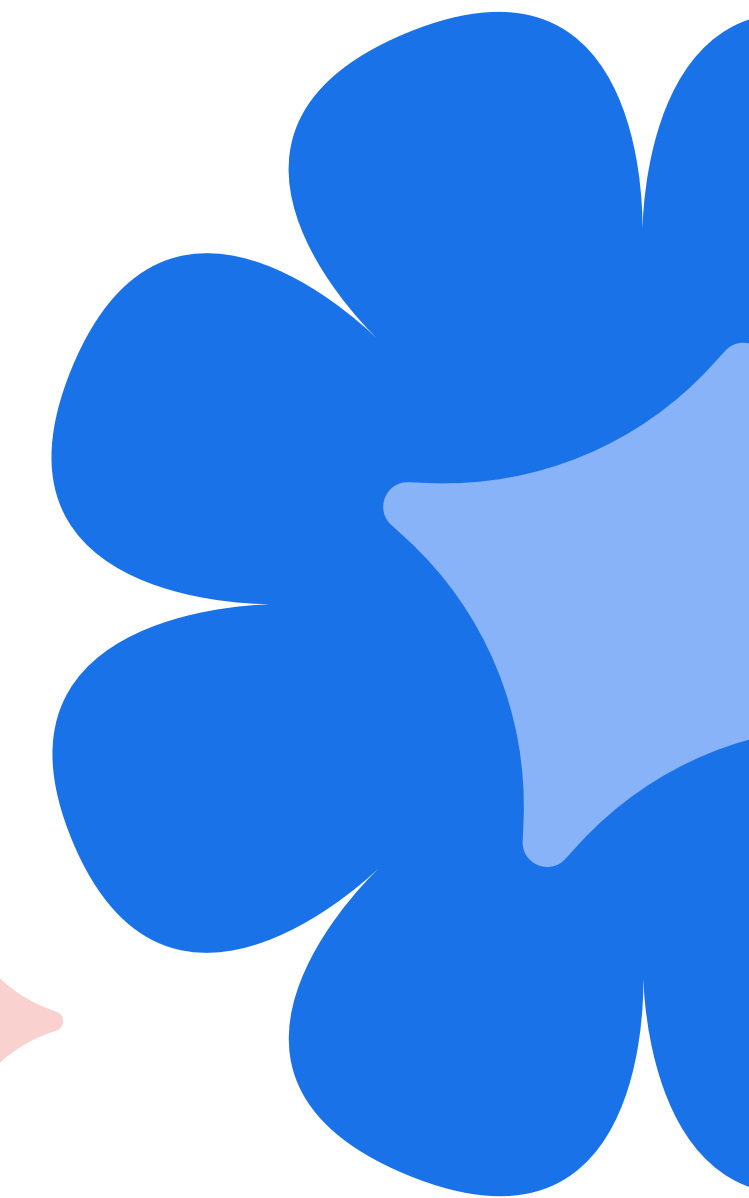
Quick reference

Choosing workflow agents

Situation	Use this
Steps depend on each other	SequentialAgent
Steps are independent	ParallelAgent
Need to repeat until done	LoopAgent

Parallel state coordination

```
Python
# ALWAYS use distinct keys for parallel agents
agent_a = LlmAgent(output_key='result_a', ...)
agent_b = LlmAgent(output_key='result_b', ...)
```



Key points to remember

- Distinct **output_key** for parallel agents (prevents conflicts)
- Aggregator after ParallelAgent to combine results
- Workflow agents before custom agents
- A2A for cross-service only

Resources

Primary references

- 📖 Workflow Agents Overview: Complete guide to workflow agents
- 📖 SequentialAgent: Ordered execution patterns
- 📖 ParallelAgent: Concurrent execution patterns
- 📖 LoopAgent: Iterative execution patterns
- 📖 Custom Agents: Building custom orchestration logic
- 🔗 A2A Protocol: Agent-to-Agent communication standard

From the community

- 📖 Fun with agentic workflows in ADK: Practical exploration of Sequential, Parallel, and Loop workflows with code examples (part 1)
- 📖 Building custom Agents from scratch using Google's ADK - When and how to build custom agents beyond workflow agents (part 3)
- 🤖 A2A_ADK_MCP: Working repository demonstrating A2A Protocol integration with ADK (Part 4)
- 🎧 A2A + ADK: The Multi-Agent AI System | Google I/O Podcast: Official Google discussion on A2A and distributed agents (part 4)
- 🎥 How to use Sessions & State in Google ADK: Video tutorial on state management for multi-agent workflows (part 2)

❓ Questions or feedback?
Share in the [ADK Community Forum](#)